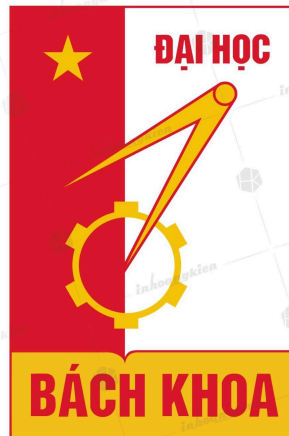


HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



CAPSTONE PROJECT

Real-Time Weather Data Analysis and Forecasting in Vietnam

Group 10

Trần Thành Vinh – 20225539

Vũ Hoàng Nhật Anh – 20225471

Nguyễn Hữu Công – 20225476

Nguyễn Thành Minh – 20225450

Đỗ Trần Gia Bách – 20225472

Instructor:

Dr. Trần Việt Trung

Faculty:

School of Information and Technology

HÀ NỘI, 12/2025
TABLE OF CONTENTS

CHAPTER 1. PROBLEM DEFINITION.....	1
1.1 Selected problem.....	1
1.2 Analysis of the Problem’s Suitability for Big Data.....	1
1.3 Scope and limitations.....	2
1.3.1 Scope.....	2
1.3.2 Limitations.....	2
CHAPTER 2. ARCHITECTURE AND DESIGN.....	3
2.1 Overall Architecture.....	3
2.2 Detailed components and their roles.....	3
CHAPTER 3. IMPLEMENTATION DETAILS.....	7
3.1 Source code.....	7
3.1.1 Reading from data sources.....	7
3.1.2 Write to Kafka.....	9
3.1.3 Write to S3.....	9
3.1.4 Bronze Stream.....	10
3.1.5 Sliver-Gold Batch.....	11
3.1.6 ML inference.....	11
3.2 Environment-specific configuration files.....	12
3.3 Deployment strategy.....	19
CHAPTER 4. LESSONS LEARNED.....	23
4.1 Data Ingestion.....	23
4.1.1 Problem description.....	23
4.1.2 Approaches tried.....	23
4.1.3 Final solution.....	23
4.1.4 Key takeaways.....	23
4.2 Data Processing with Spark.....	23
4.2.1 Problem description.....	23
4.2.2 Approaches tried.....	23
4.2.3 Final solution.....	24
4.2.4 Key takeaways.....	24
4.3 Stream Processing.....	24

4.3.1	Problem description.....	24
4.3.2	Approaches tried.....	24
4.3.3	Final solution.....	24
4.3.4	Key takeaways.....	24
4.4	Data Storage.....	24
4.4.1	Problem description.....	24
4.4.2	Approaches tried.....	25
4.4.3	Final solution.....	25
4.4.4	Key takeaways.....	25
4.5	System Integration.....	25
4.5.1	Problem Description.....	25
4.5.2	Approaches tried.....	25
4.5.3	Final solution.....	26
4.5.4	Key takeaways.....	26
4.6	Performance Optimization.....	26
4.6.1	Problem Description.....	26
4.6.2	Approaches tried.....	26
4.6.3	Final solution.....	26
4.6.4	Key takeaways.....	26
4.7	Monitoring and Debugging.....	26
4.7.1	Problem Description.....	26
4.7.2	Approaches tried.....	26
4.7.3	Final solution.....	26
4.7.4	Key takeaways.....	27
4.8	Scaling.....	27
4.8.1	Problem Description.....	27
4.8.2	Approaches tried.....	27
4.8.3	Final solution.....	27
4.8.4	Key takeaways.....	27
4.9	Data Quality & Testing.....	28
4.9.1	Problem description.....	28
4.9.2	Approaches tried.....	28
4.9.3	Final solution.....	28

4.9.4	Key takeaways.....	28
4.10	Fault Tolerance.....	28
4.10.1	Problem Description.....	28
4.10.2	Approaches tried.....	28
4.10.3	Final solution.....	28
4.10.4	Key takeaways.....	28

CHAPTER 1. PROBLEM DEFINITION

1.1 Selected problem

Predicting weather is important and can be highly beneficial in many cases, especially for countries that is heavily dependent on the weather. However, traditional forecasting methods often struggle with limited spatial resolution—providing predictions that are too broad for specific districts—and high latency, where data processing takes too long during time-critical situations. Therefore, developing a real-time, scalable system for weather forecasting in Vietnam is essential.

The core problem is the ability to integrate diverse, high-frequency data streams into a unified model that can provide accurate, localized, and immediate predictions for future temperature.

1.2 Analysis of the Problem's Suitability for Big Data

Weather forecasting is a quintessential "Big Data" problem because it fits the 5Vs framework perfectly:

Volume: Historical data can be very large (up to 300000 rows). Read-write operations needs to be handle quickly.

Velocity: Data from OpenWeatherAPI arrives in real-time or near-real time. To be useful for forecasting, this data must be ingested and analyzed in seconds.

Variety: The data is highly heterogeneous, including:

- **OpenWeatherAPI:** Numerical readings (temp, pressure, humidity) from APIs. These APIs read from IoT sensors and originally JSON-like.
- **Open-Meteo:** Also numerical readings, but it is different in terms of the structure. Data from this source is also JSON-like, but contain different fields.

The data from both sources need to be written in a single format (parquet).

Veracity: Weather data is often "noisy" or inconsistent due to sensor malfunctions during storms. Big Data tools are required to perform real-time data cleaning. Also, since the data comes from 2 different sources, there are columns that may not appear in both sources.

Value: Accurate real-time forecasting has massive socio-economic value, potentially saving lives and reducing the billions of VND in annual damages to Vietnam's agriculture and infrastructure.

1.3 Scope and limitations

1.3.1 Scope

Geographical Focus: The project analyze 34 cities in Vietnam

Data Sources: Open sources like OpenWeatherAPI and Open-Meteo. The data are harvested using API calls.

Temporal Scale: Focus on short-term (24-hour data), predicting the next hour data (temperature). The data can be extended to much larger time windows, with the historical data saved on S3.

Target Output: A dashboard for weather analysis, with the information of next hour temperature.

1.3.2 Limitations

Hardware Costs: Processing high-resolution Numerical Weather Prediction (NWP) models requires significant GPU/CPU power and expensive cloud infrastructure.

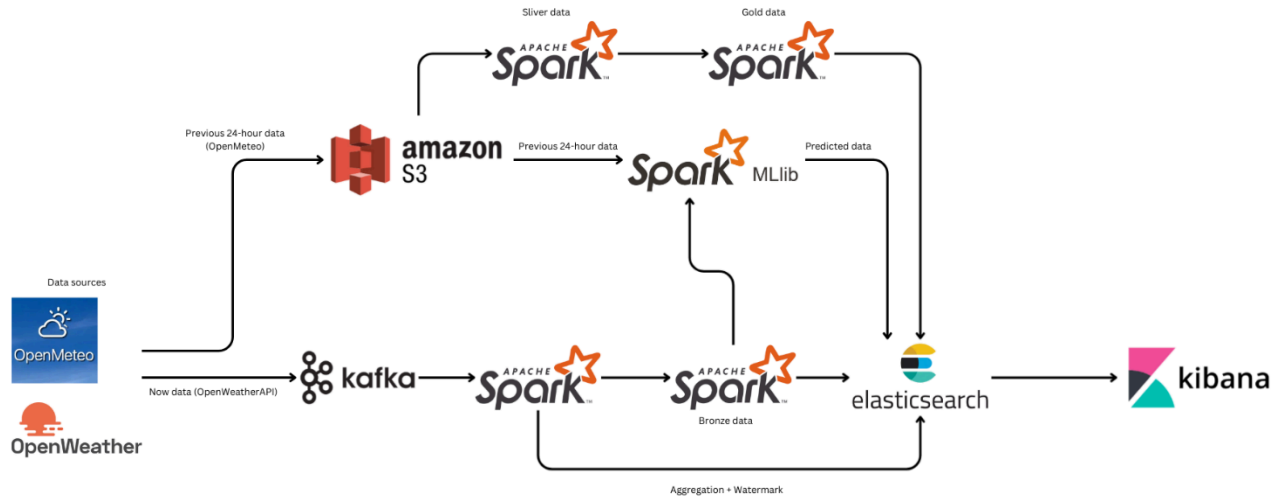
Data Gaps: Some areas do not have sensors to track the weather.

Predictability Limits: Even with Big Data, the "Butterfly Effect" (chaos theory) means that atmospheric behaviours for further time periods remains inherently unpredictable.

CHAPTER 2. ARCHITECTURE AND DESIGN

2.1 Overall Architecture

This project follows Lambda Architecture. The detailed pipeline is shown below:



2.2 Detailed components and their roles

- Data sources: (OpenWeather and OpenMeteo)

These sources provide APIs for weather data. Our system is NOT capable of reading data directly from IoT sensors.

The reason our system needs 2 sources is: we need both historical data (for ML) and current weather data (for showing). Weather data from OpenMeteo provides us with hourly data (in other words, weather data in an hour). Whereas weather data from OpenWeather provides us with current weather data (down to seconds).

An example to see the difference between these 2 sources is: if you call both APIs at 7:03:46, OpenMeteo returns 7:00:00 data, OpenWeather returns 7:03:46 data.

An example of **OpenWeather API**:

```
[2025-12-28 13:00:35.980135] Collecting weather data for Hanoi...
Data from OPENWEATHER:
{'base': 'stations',
 'clouds': {'all': 100},
 'cod': 200,
 'coord': {'lat': 21.0245, 'lon': 105.8412},
 'dt': 1766901228,
 'id': 1581130,
 'main': {'feels_like': 19.54,
          'grnd_level': 1017,
          'humidity': 57,
          'pressure': 1018,
          'sea_level': 1018,
          'temp': 20,
          'temp_max': 20,
          'temp_min': 20},
 'name': 'Hanoi',
 'sys': {'country': 'VN',
         'id': 9308,
         'sunrise': 1766878327,
         'sunset': 1766917427,
         'type': 1},
 'timezone': 25200,
 'visibility': 10000,
 'weather': [{'description': 'overcast clouds',
               ...
```

An example of OpenMeteo API:

```
{
  "latitude": 21.0,
  "longitude": 105.875,
  "generationtime_ms": 0.1646280288696289,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 19.0,
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "°C",
    "relative_humidity_2m": "%",
    "surface_pressure": "hPa",
    "cloud_cover": "%",
    "wind_speed_10m": "km/h",
    "wind_direction_10m": "°",
    "weather_code": "wmo code"
  },
  "hourly": {
    "time": [
      "2025-12-27T00:00", "2025-12-27T01:00", "2025-12-27T02:00",
      ...
    ],
    "temperature_2m": [
      15.0, 15.4, 16.1, ...
    ],
    ...
  }
}
```

- Amazon S3:

Amazon S3 serves as a storage for historical data. Duplication and data partition is handled automatically by AWS S3.

- **Kafka:**

Kafka handles stream data. In this case, stream data is the current weather data from OpenWeather API.

- **Spark ML:**

Spark ML component handles ML model inference. Keep in mind that in our pipeline we don't retrain the model, as training the model take a year worth of data (one-year data). Considering the fact that the pipeline if ran in real life scenario will be run everyday, retraining everyday is not efficient.

Choosing a longer interval for retraining is not possible since retraining can be taken outside of the pipeline.

We chose Gradient Boosting Tree for inference. The features are 24-hour lags and temperature, humidity and pressure of the current hour to predict the next-hour weather.

- **Spark (Bronze data):**

Spark component process raw data (bronze data) writes to ES for showing (if used). Since the format of the data from OpenWeather API is JSON, so the format output of this component is JSON-like.

- **Spark (Sliver data):**

This component drops null columns, normalize data types of schema's fields and add 24-hour average temperature. The output of this component is pushed to Spark Gold data handling component.

- **Spark (Gold data):**

This component add min, max, range, wind_gust, ... This is for SHOWING only and is not used in ML (for instant inference).

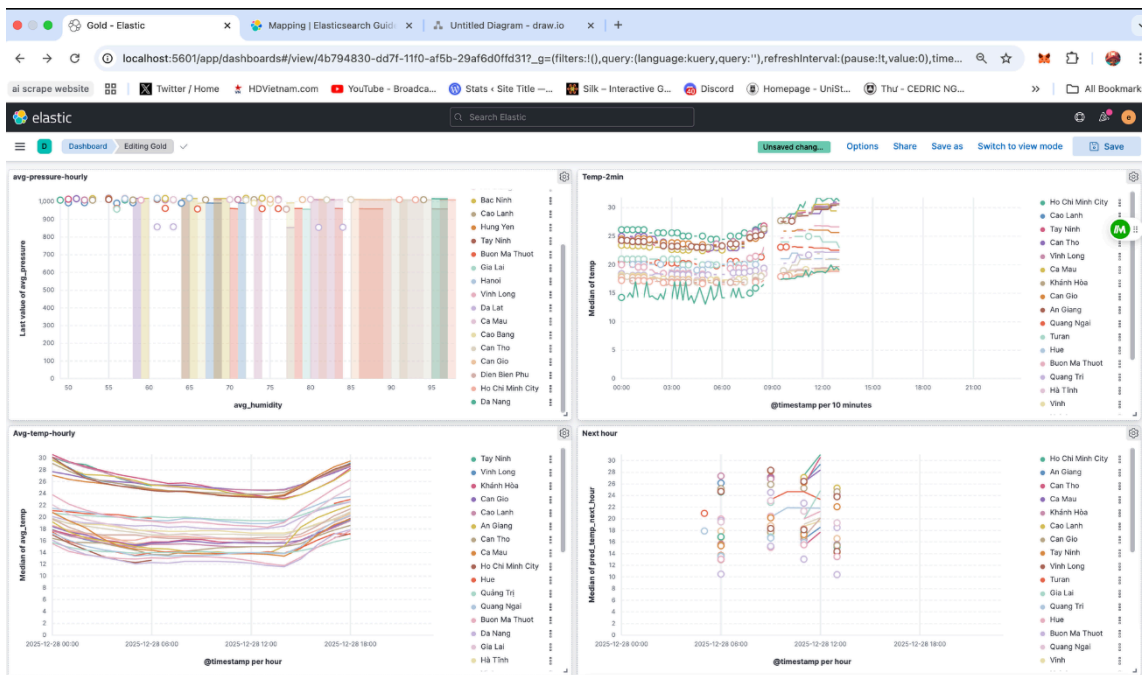
- **Elasticsearch:**

Elasticsearch serves as the serving layer for our system. Elasticsearch resembles somewhat MongoDB, since it supports unstructured data like JSON. However, we processed all data into DataFrame (structured, because of ML) before writing to Elasticsearch.

- **Kibana:**

Kibana works with Elasticsearch. Kibana retrieves data from Elasticsearch to show it to the users using Kibana Query Language (KQL).

Kibana runs as an app to show the dashboard. A dashboard page can be seen below:



- Kubernetes:

Kubernetes provides a unified orchestration layer that enables streaming, batch, and ML workloads to run on the same infrastructure while ensuring resource scheduling, isolation, and fault recovery for services such as Spark, Kafka, and Elasticsearch.

It simplifies deployment and lifecycle management through declarative manifests and operator-based automation, including Strimzi, Spark Operator, and Elastic Operator.

PodCIDR:	10.244.0.0/24					
PodCIDRs:	10.244.0.0/24					
ProviderID:	kind://docker/data-platform/data-platform-control-plane					
Non-terminated Pods:	(29 in total)					
Namespace	Name	CPU Requests	CPU Limits	Memory Requests	Memory Limits	Age
data	kafka-entity-operator-5968477844-cnhzk	0 (0%)	0 (0%)	0 (0%)	0 (0%)	17h
data	kafka-kafka-pool-0	0 (0%)	0 (0%)	0 (0%)	0 (0%)	109m
data	kafka-kafka-pool-1	0 (0%)	0 (0%)	0 (0%)	0 (0%)	113m
data	kafka-kafka-pool-2	0 (0%)	0 (0%)	0 (0%)	0 (0%)	113m
data	strimzi-cluster-operator-8457d7566-hx2cn	200m (1%)	1 (8%)	384Mi (3%)	384Mi (3%)	9d
data	weather-kafka-producer-5d7b968f59-f5c6p	0 (0%)	0 (0%)	0 (0%)	0 (0%)	3d
default	bronze-to-es-batch-97b6f89b644cf7c-exec-1	1 (8%)	0 (0%)	1433Mi (11%)	1433Mi (11%)	41s
default	bronze-to-es-batch-97b6f89b644cf7c-exec-2	1 (8%)	0 (0%)	1433Mi (11%)	1433Mi (11%)	41s
default	bronze-to-es-batch-driver	1 (8%)	0 (0%)	1433Mi (11%)	1433Mi (11%)	44s
default	checkpoint-debug	0 (0%)	0 (0%)	0 (0%)	0 (0%)	8d
default	silver-debug	0 (0%)	0 (0%)	0 (0%)	0 (0%)	8d
default	spark-debug	0 (0%)	0 (0%)	0 (0%)	0 (0%)	16h
default	weather-streaming-to-bronze-b311d79b638cfba4-exec-1	1 (8%)	0 (0%)	1433Mi (11%)	1433Mi (11%)	3h30m
default	weather-streaming-to-bronze-b311d79b638cfba4-exec-2	1 (8%)	0 (0%)	1433Mi (11%)	1433Mi (11%)	3h30m
default	weather-streaming-to-bronze-driver	1 (8%)	0 (0%)	1433Mi (11%)	1433Mi (11%)	3h30m
elastic-system	elastic-operator-0	100m (0%)	1 (8%)	150Mi (1%)	161 (8%)	9d
kube-system	coredns-66bc5c9577-4nllb	100m (0%)	0 (0%)	70Mi (0%)	170Mi (1%)	9d
kube-system	coredns-66bc5c9577-lx5d5	100m (0%)	0 (0%)	70Mi (0%)	170Mi (1%)	9d
kube-system	etcd-data-platform-control-plane	100m (0%)	0 (0%)	100Mi (0%)	0 (0%)	9d
kube-system	kindnet-hj7ln	100m (0%)	100m (0%)	50Mi (0%)	50Mi (0%)	9d
kube-system	kube-apiserver-data-platform-control-plane	250m (2%)	0 (0%)	0 (0%)	0 (0%)	9d
kube-system	kube-controller-manager-data-platform-control-plane	200m (1%)	0 (0%)	0 (0%)	0 (0%)	9d
kube-system	kube-proxy-zcgt	0 (0%)	0 (0%)	0 (0%)	0 (0%)	9d
kube-system	kube-scheduler-data-platform-control-plane	100m (0%)	0 (0%)	0 (0%)	0 (0%)	9d
local-path-storage	local-path-provisioner-7b8c8dbd6-54m6x	0 (0%)	0 (0%)	0 (0%)	0 (0%)	9d
observability	es-weather-es-default-0	250m (2%)	1 (8%)	1Gi (8%)	161 (8%)	9d
observability	kibana-weather-kb-76fd7567b8-f4xv6	200m (1%)	1 (8%)	512Mi (4%)	1Gi (8%)	8d
spark-operator	spark-operator-controller-5fbfcb9d47-wshds	0 (0%)	0 (0%)	0 (0%)	0 (0%)	9d
spark-operator	spark-operator-webhook-55c6f74b59-xn4vm	0 (0%)	0 (0%)	0 (0%)	0 (0%)	9d

CHAPTER 3. IMPLEMENTATION DETAILS

3.1 Source code

This section we will show only important parts of our source code. This is because all the source code files is too long, and you can take a look at it on our GitHub repository.

3.1.1 Reading from data sources

- City list (list of cities – 34 of Vietnam. Some we have to approximate using a town in that region):

```
CITY_LIST = [  
    "An Giang",  
    "Bac Ninh",  
    "Ca Mau",  
    "Cao Bang",  
    "Can Tho",  
    "Da Nang",  
    "Bun Ma Thuot", # approximate Đắk Lắk  
    "Dien Bien Phu", # approximate Điện Biên  
    "Can Gio", # approximate Đồng Nai  
    "Cao Lanh", # approximate Đồng Tháp  
    "Gia Lai",  
    "Hanoi",  
    "Hà Tĩnh",  
    "Haiphong",  
    "Ho Chi Minh City",  
    "Hung Yen",  
    "Khánh Hòa",  
    "Lai Chau",  
    "Da Lat", # approximate Lâm Đồng  
    "Lang Son",  
    "Lao Cai",  
    "Vinh", # approximate Nghệ An  
    "Ninh Binh",  
    "Phu Tho",  
    "Quang Ngai",  
    "Cam Pha Mines", # approximate Quảng Ninh  
    "Quảng Trị",  
    "Son La",  
]
```

- Calling OpenWeather API:

```

# ===== WEATHER =====
def fetch_weather(city: str) -> dict | None:
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {
        "q": city,
        "appid": API_KEY,
        "units": "metric",
    }

    try:
        r = requests.get(url, params=params, timeout=10)
        if r.status_code != 200:
            print(f"[WARN] API {r.status_code} for {city}")
            return None

        data = r.json()

        # Align event_time to hour (UTC)
        event_time = (
            datetime.now(timezone.utc)
            .replace(minute=0, second=0, microsecond=0)
            .isoformat()
        )

        weather = data.get("weather", [{}])[0]
        main = data.get("main", {})
        wind = data.get("wind", {})
        clouds = data.get("clouds", {})

```

- Calling OpenMeteo API:

```

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": coords["lat"],
    "longitude": coords["lon"],
    "hourly": (
        "temperature_2m,relative_humidity_2m,surface_pressure,"
        "cloud_cover,wind_speed_10m,wind_direction_10m,weather_code"
    ),
    "start_date": start_utc.strftime("%Y-%m-%d"),
    "end_date": now_utc.strftime("%Y-%m-%d"),
}

res = requests.get(url, params=params, timeout=10)
res.raise_for_status()

hourly = res.json()["hourly"]

rows = []
for i, t in enumerate(hourly["time"]):
    # API trả ISO string → datetime (UTC)
    ts_utc = datetime.fromisoformat(t)
    # ...

```

3.1.2 Write to Kafka

```
def send_to_kafka(record: dict):
    producer.produce(
        topic=TOPIC,
        key=str(record["city"]).encode(),
        value=json.dumps(record).encode(),
    )
    producer.poll(0)

def main():
    print(f"🚀 Weather producer started → topic={TOPIC}")

    while True:
        for city in CITY_LIST:
            record = fetch_weather(city)
            if record:
                send_to_kafka(record)

        producer.flush()
        time.sleep(POLL_INTERVAL)
```

3.1.3 Write to S3

- Data before writing to S3 needs to be merged and drop duplicate:

```
# ===== TIMESTAMP FIX (CRITICAL) =====
df["timestamp"] = pd.to_datetime(df["timestamp"])
df["timestamp"] = df["timestamp"].dt.floor("s") # remove nanoseconds

df.drop_duplicates(subset=["city", "timestamp"], inplace=True)

key = f"{SILVER_PREFIX}/{city}.parquet"

# ===== MERGE WITH EXISTING =====
try:
    obj = s3.get_object(Bucket=S3_BUCKET, Key=key)
    old_df = pd.read_parquet(io.BytesIO(obj["Body"].read()))

    old_df["timestamp"] = pd.to_datetime(old_df["timestamp"]).dt.floor("s")

    df = pd.concat([old_df, df], ignore_index=True)
    df.drop_duplicates(subset=["city", "timestamp"], inplace=True)

except ClientError as e:
    if e.response["Error"]["Code"] != "NoSuchKey":
        raise

# ===== WRITE PARQUET (SPARK SAFE) =====
```

- Then it is written to S3:

```

# ===== WRITE PARQUET (SPARK SAFE) =====
out = io.BytesIO()

table = pa.Table.from_pandas(
    df,
    preserve_index=False,
)

pq.write_table(
    table,
    out,
    coerce_timestamps="us", # KEY LINE
    allow_truncated_timestamps=True, # avoid ns overflow
)

s3.put_object(
    Bucket=S3_BUCKET,
    Key=key,
    Body=out.getvalue(),
)

print(f"Silver updated: s3://{S3_BUCKET}/{key}")

```

3.1.4 Bronze Stream

- Read from kafka:

```

# READ KAFKA

raw = (
    spark.readStream.format("kafka")
        .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP)
        .option("subscribe", TOPIC)
        .option("startingOffsets", "latest")
        .option("failOnDataLoss", "false")
        .load()
)

parsed = raw.selectExpr("CAST(value AS STRING) AS value").withColumn(
    "json", from_json(col("value"), schema)
)

# BRONZE STREAM (RAW, IMMUTABLE, FULL PAYLOAD)

bronze_df = parsed.select("json.*")

q_bronze = (
    bronze_df.writeStream.format("parquet")
        .outputMode("append")
        .option("path", BRONZE_PATH)
        .option("checkpointLocation", f"{CHECKPOINT_BASE}/bronze")
        .start()
)

```

- Normalize data:

```

# Normalize timestamp + ES id

df = (
    df
    # REAL EVENT TIME
    .withColumn("event_time", to_timestamp(from_unixtime(col("dt"))))
    # ES DATE FORMAT
    .withColumn("@timestamp", date_format(col("event_time"), "yyyy-MM-dd'T'HH:mm:ss"))
    # Idempotent ID
    .withColumn("doc_id", concat_ws("_", col("city"), col("dt")))
)

```

3.1.5 Sliver-Gold Batch

- Processing data from S3 to sliver:

```

126 /
127
128 silver = base.withColumn(
129     "avg_temp_24h",
130     avg("temp").over(w_24h),
131 )
132
133
134 # PERFORMANCE OPTIMIZATION
135
136 silver = silver.repartition("city")
137 silver.cache()
138
139 count = silver.count()
140 print(f"🔍 Silver record count: {count}")
141
142 silver.groupBy("city").count().show(truncate=False)
143
144
145 # WRITE SILVER (PARTITIONED BY CITY)
146 (silver.write.mode("overwrite").partitionBy("city").parquet(SILVER_PATH))
147
148 print("✅ Silver job completed successfully")
149 spark.stop()

```

3.1.6 ML inference

- Training code (not in the pipeline):

```

feature_inputs = ["temp", "humidity", "pressure"] + lag_cols
assembler = VectorAssembler(inputCols=feature_inputs, outputCol="features")

# 7. Train Global Model
gbt = GBTRegressor(featuresCol="features", labelCol="target_future", maxIter=20)
pipeline = Pipeline(stages=[assembler, gbt])

print("Training Global GBT Model...")
model = pipeline.fit(train_data)

```

- Inference:

```

df = df.withColumn(f"temp_lag_{i}", F.lag("temp", i).over(w))

# Target (not used for inference, but required by pipeline schema)
df = df.withColumn("target_future", F.lead("temp", 1).over(w)).dropna()

# LOAD MODEL (ONCE - DRIVER ONLY)

if not os.path.exists(MODEL_PATH):
    raise RuntimeError(f"❌ Model path not found: {MODEL_PATH}")

print(f"📦 Loading ML model from: {MODEL_PATH}")

model = PipelineModel.load(MODEL_PATH)

print(f"✅ Model loaded successfully")

# INFERENCE

pred = model.transform(df)

# KEEP LATEST PER CITY

pred = pred.withColumn("max_ts", F.max("timestamp").over(Window.partitionBy("city")))

final = pred.filter(F.col("timestamp") == F.col("max_ts")).select(
    F.col("city"),
    F.col("timestamp").alias("@timestamp"),
    F.col("prediction").alias("pred_temp_next_hour"),
)

```

3.2 Environment-specific configuration files

This section we will show important .yaml files that are important to our k8s deployment.

- Producer (Data sources readers):

Weather-producer.yaml (for kafka):

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: weather-kafka-producer
  namespace: data
spec:
  replicas: 1
  selector:
    matchLabels:
      app: weather-kafka-producer
  template:
    metadata:
      labels:
        app: weather-kafka-producer
    spec:
      containers:
        - name: producer
          image: weather-kafka-producer:latest
          imagePullPolicy: IfNotPresent
          env:
            - name: KAFKA_BROKER
              value: kafka-kafka-bootstrap.data.svc.cluster.local:9092
            - name: KAFKA_TOPIC
              value: weather-raw
            - name: OPENWEATHER_API_KEY
              valueFrom:
                secretKeyRef:
                  name: openweather-secret
                  key: api-key
            - name: POLL_INTERVAL
              value: "60"

```

Weather-history-cronjob.yaml (for historical S3 data):


```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: weather-history-to-s3
  namespace: data
spec:
  schedule: "*/45 * * * *" # mỗi 45 phút
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 2

  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure

          containers:
            - name: s3-history
              image: weather-s3-history:latest
              imagePullPolicy: IfNotPresent

              env:
                - name: S3_BUCKET_NAME
                  value: hust-bucket-storage

                - name: AWS_ACCESS_KEY_ID
                  valueFrom:
                    secretKeyRef:
                      name: aws-secret
                      key: access-key

```

- Kafka:

Kafka-nodepool.yaml:

```

apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: kafka-pool
  namespace: data
  labels:
    strimzi.io/cluster: kafka
spec:
  replicas: 3
  roles:
    - controller
    - broker
  storage:
    type: ephemeral

```

Kafka.yaml:

```
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: kafka
  namespace: data
spec:
  kafka:
    version: 4.0.1
    metadataVersion: 4.0-IV1

    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false

    config:
      num.partitions: 3
      default.replication.factor: 3
      min.insync.replicas: 2

      offsets.topic.replication.factor: 3
      transaction.state.log.replication.factor: 3
      transaction.state.log.min.isr: 2

      auto.create.topics.enable: "false"

  entityOperator:
    topicOperator: {}
    userOperator: {}
```

- S3:

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: weather-history-to-s3
  namespace: data
spec:
  schedule: "*/45 * * * *" # mỗi 45 phút
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 2

  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure

          containers:
            - name: s3-history
              image: weather-s3-history:latest
              imagePullPolicy: IfNotPresent

              env:
                - name: S3_BUCKET_NAME
                  value: hust-bucket-storage

                - name: AWS_ACCESS_KEY_ID
                  valueFrom:
                    secretKeyRef:
                      name: aws-secret
                      key: access-key

```

- Bronze:

Bronze-to-es.yaml:

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: bronze-to-es-batch-cron
  namespace: default
spec:
  schedule: "*/2 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  cat <<'EOF' | kubectl apply -f -
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:

```

There are many other bronze .yaml files, please refer to the GitHub repository.

- Sliver:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: weather-s3-to-silver-cron
  namespace: default
spec:
  schedule: "0 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-silver-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  kubectl create -f - <<'EOF'
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:
```

- Gold:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: silver-to-gold-es-cron
  namespace: default
spec:
  schedule: "10 * * * *" # chạy sau silver 10 phút
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-gold-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  kubectl create -f - <<'EOF'
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:
```

- ML:

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: weather-ml-inference-cron
  namespace: default
spec:
  schedule: "0 * * * *" # 🕒 mỗi giờ
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-ml-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  kubectl create -f - <<'EOF'
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:
                    name: weather-ml-inference

```

- Elasticsearch:

```

apiVersion: elasticsearch.k8s.elastic.co/v1
kind: Elasticsearch
metadata:
  name: es-weather
  namespace: observability
spec:
  version: 7.17.9
  http:
    tls:
      selfSignedCertificate:
        disabled: true
  nodeSets:
    - name: default
      count: 1
      config:
        node.store.allow_mmap: false
      podTemplate:
        spec:
          containers:
            - name: elasticsearch
              resources:
                requests:
                  memory: 1Gi
                  cpu: 250m
                limits:
                  memory: 1Gi
                  cpu: 1

```

- Kibana

```

apiVersion: kibana.k8s.elastic.co/v1
kind: Kibana
metadata:
  name: kibana-weather
  namespace: observability
spec:
  version: 7.17.9
  count: 1

  elasticsearchRef:
    name: es-weather

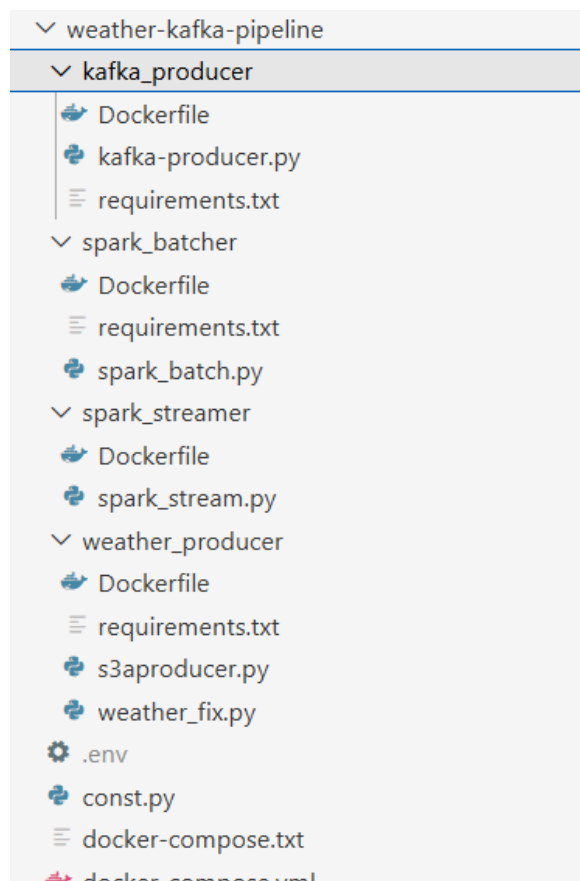
  http:
    tls:
      selfSignedCertificate:
        disabled: true # dŕ port-forward, debug

  podTemplate:
    spec:
      containers:
        - name: kibana
          resources:
            requests:
              cpu: 200m
              memory: 512Mi
            limits:
              cpu: 1
              memory: 1Gi

```

3.3 Deployment strategy

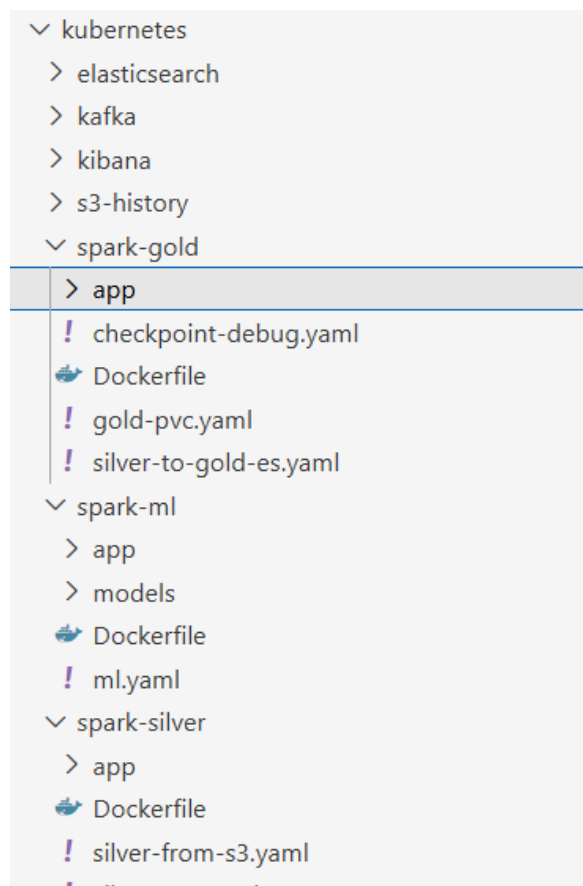
We first deploy on docker using many Dockerfiles, with a docker-compose.yml file to run everything. Spark components are run as Spark jobs with no Master or Worker nodes.



Here are 2 containers running Spark jobs in the early stages of the project:

```
spark-streamer:
  build:
    context: ./spark_streamer
  # volumes:
  #   - ./spark_stream.py:/app/spark_stream.py
  container_name: spark-streamer
  depends_on:
    - kafka
    - elasticsearch
  networks:
    - weather-net
  ▷ Run Service
spark-batcher:
  build:
    context: ./spark_batcher      # Folder containing Dockerfile + script
  container_name: spark-batcher
  depends_on:
    - kafka
    - elasticsearch
  env_file:
    - .env
  networks:
    - weather-net
```

To push to k8s, we have to change Dockerfiles to yaml files and put all code logic in app directories. There are Dockerfiles, still, but those are for building the image and load it to k8s, but NOT for configuration.



For example, for spark-gold component, app/ directory is for code, .yaml files are for configuration, and the Dockerfile is for building an image and loading it to k8s.

Checkpoint-debug.yaml:

```
apiVersion: v1
kind: Pod
metadata:
  name: checkpoint-debug
spec:
  containers:
    - name: debug
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: checkpoint
          mountPath: /checkpoint
  volumes:
    - name: checkpoint
      persistentVolumeClaim:
        claimName: spark-checkpoint-pvc
```

Dockerfile:

```

FROM apache/spark:3.3.3 (last pushed 2 years ago)

USER root

# =====
# 1. Thư mục bắt buộc cho SparkSubmit + spark.jars.packages
# =====
RUN mkdir -p /tmp/.ivy/cache /tmp/.ivy/jars /tmp/spark-upload && \
    chown -R 185:185 /tmp/.ivy /tmp/spark-upload

RUN mkdir -p /opt/spark/.ivy/cache /opt/spark/.ivy/jars && \
    chown -R 185:185 /opt/spark/.ivy

# =====
# 2. Elasticsearch Spark connector (MATCH Spark 3.3.x)
# =====
WORKDIR /opt/spark
RUN curl -fLo jars/elasticsearch-spark-30_2.12-8.11.3.jar \
    https://repo1.maven.org/maven2/org/elasticsearch/elasticsearch-spark-30_2.12/8.1

# =====
# 3. App code
# =====
WORKDIR /app
COPY app /app
RUN chown -R 185:185 /app

USER 185

```

Kubernetes:

The system is deployed using a **single-node Kubernetes cluster**, where a single node simultaneously fulfills both control plane and worker responsibilities.

Specifically, this node hosts:

- **Control Plane components**, including the Kubernetes API Server, Scheduler, and Controller Manager
- **Worker node functions**, responsible for running all application workloads

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER
data-platform-control-plane	Ready	control-plane	9d	v1.34.0	172.19.0.2	<none>	Debian GNU/Linux 12 (bookworm)	6.10.14-linuxkit	containerd://2.1.3

```

Name: data-platform-control-plane
Roles: control-plane
Labels: beta.kubernetes.io/arch=arm64
        beta.kubernetes.io/os=linux
        kubernetes.io/arch=arm64
        kubernetes.io/hostname=data-platform-control-plane
        kubernetes.io/os=linux
        node-role.kubernetes.io/control-plane=
        node.alpha.kubernetes.io/ttl: 0
Annotations: volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp: Thu, 18 Dec 2025 17:08:39 +0700
Taints: <none>
Unschedulable: false
Lease:
  HolderIdentity: data-platform-control-plane
  AcquireTime: <unset>
  RenewTime: Sun, 28 Dec 2025 16:32:42 +0700
Conditions:
  Type          Status  LastHeartbeatTime  LastTransitionTime  Reason              Message
  MemoryPressure False   Sun, 28 Dec 2025 16:31:21 +0700  Sun, 28 Dec 2025 14:03:56 +0700  KubeletHasSufficientMemory  kubelet has sufficient memory available
  DiskPressure   False   Sun, 28 Dec 2025 16:31:21 +0700  Sun, 28 Dec 2025 14:03:56 +0700  KubeletHasNoDiskPressure    kubelet has no disk pressure
  PIDPressure    False   Sun, 28 Dec 2025 16:31:21 +0700  Sun, 28 Dec 2025 14:03:56 +0700  KubeletHasSufficientPID     kubelet has sufficient PID available
  Ready          True    Sun, 28 Dec 2025 16:31:21 +0700  Sun, 28 Dec 2025 14:18:57 +0700  KubeletReady                kubelet is posting ready status
Addresses:
  InternalIP: 172.19.0.2
  Hostname: data-platform-control-plane
Capacity:
  cpu: 12
  ephemeral-storage: 1055761844Ki
  hugepages-1Gi: 0
  hugepages-2Mi: 0
  hugepages-32Mi: 0
  hugepages-64Ki: 0
  memory: 12235624Ki
  pods: 110
Allocatable:
  cpu: 12
  ephemeral-storage: 1055761844Ki
  hugepages-1Gi: 0
  hugepages-2Mi: 0
  hugepages-32Mi: 0
  hugepages-64Ki: 0
  memory: 12235624Ki
  pods: 110
System Info:
  Machine ID: 35ac27dd5a014891861f7de30a71ba55
  System UUID: 35ac27dd5a014891861f7de30a71ba55
  Boot ID: b02bc92d-e159-4c43-9240-5dc5f98a8cc5
  Kernel Version: 6.10.14-linuxkit
  OS Image: Debian GNU/Linux 12 (bookworm)
  Operating System: linux
  Architecture: arm64
  Container Runtime Version: containerd://2.1.3

```

To ensure clear logical separation and manageable system organization, system components are deployed across multiple Kubernetes namespaces.

```

(big-data) minh@MacBook-Pro-cua-Minh kubernetes % kubectl get ns -o name
namespace/data
namespace/default
namespace/elastic-system
namespace/kafka
namespace/kube-node-lease
namespace/kube-public
namespace/kube-system
namespace/local-path-storage
namespace/observability
namespace/spark-operator

```

CHAPTER 4. LESSONS LEARNED

4.1 Data Ingestion

4.1.1 Problem description

Data comes from 2 different sources: OpenWeatherAPI and Open-Meteo. The structure of data from these sources are not similar. In fact, they are quite different and requires us to handle (drop unnecessary columns).

Also, we need to flatten the data from OpenWeatherAPI (since it is originally JSON-like)

4.1.2 Approaches tried

Drop unnecessary columns: This is because the actual amount of data required for training ML model (keep in mind that without the predicting feature, we just need a simple stream pipeline to show the current weather)

4.1.3 Final solution

Some unnecessary columns are dropped. This makes the ML processing part much faster.

4.1.4 Key takeaways

Handling different data sources: Data from different APIs often have inconsistent structures, requiring careful preprocessing and normalization.

Flattening JSON: Flattening JSON data and removing unnecessary columns simplifies the dataset and prepares it for efficient ML processing.

Dropping columns: Reducing data dimensionality significantly improves processing speed, especially for real-time weather pipelines.

4.2 Data Processing with Spark

4.2.1 Problem description

Installing Spark on Docker is difficult, since it requires a lot of “other items”. Some of them are:

Dependency: Spark requires a specific version compatibility between Java (JDK), Python, and Scala. Incompatibility often leads to "Class Not Found" or "Py4J" errors. **So, a container running Spark needs JDK.**

Environment Variables: Manual configuration of SPARK_HOME, HADOOP_HOME, and PATH is prone to human error, which prevents the Spark session from initializing.

4.2.2 Approaches tried

Dependency: We use a pre-built container with JDK installed (eclipse-temurin:17-jdk).

Environment Variables: All environment variables are in the corresponding .yaml files of Spark.

4.2.3 Final solution

The solution implemented all approaches above.

4.2.4 Key takeaways

Version Alignment: Always verify that your Spark version (e.g., 3.5.0) matches the corresponding Hadoop and Java (typically JDK 8 or 11) versions.

Containerization is King: Using Docker significantly reduces "Cold Start" time for new team members joining the project.

4.3 Stream Processing

4.3.1 Problem description

Installing Zookeeper for Kafka is challenging because it requires correct coordination between multiple services and configurations. Some common issues include:

Version Compatibility: Kafka and Zookeeper must be compatible. Using mismatched versions often causes broker startup failures or connection timeouts.

Configuration Complexity: Zookeeper requires proper configuration of zoo.cfg, including dataDir, clientPort, and server settings. Incorrect paths or ports prevent Zookeeper from starting correctly.

4.3.2 Approaches tried

Version Compatibility: Use Kafka and Zookeeper versions that are officially supported together.

Configuration Management: Define all Zookeeper configurations explicitly in the Dockerfile/configuration files to avoid manual errors. (For the last version, Dockerfiles are no longer available)

4.3.3 Final solution

Zookeeper was deployed using a compatible version with Kafka, fully configured through Docker, and initialized before Kafka startup to ensure stable coordination.

4.3.4 Key takeaways

Dependency Awareness: Kafka cannot function correctly without a properly configured and running Zookeeper instance.

4.4 Data Storage

4.4.1 Problem description

Handling big time-series data requires a suitable type of data storage. Several storages have been tested.

4.4.2 Approaches tried

Avro (Apache Avro): A row-based storage format. We tested this primarily for the ingestion layer (landing zone) because it is excellent for writing data quickly and handling schema changes (e.g., if an API adds a new weather parameter). However, it performed poorly during analysis because Spark had to read the entire row even if we only needed the "Temperature" column.

ORC (Optimized Row Columnar): A columnar storage format originally designed for Apache Hive. It offered the highest compression ratios of the three. While very efficient, we found it slightly less "pluggable" with the broader Python/Data Science ecosystem compared to other formats when moving data outside of the Hadoop environment.

Parquet (Apache Parquet): A columnar storage format designed specifically for nested data. We tested this for our "Silver" and "Gold" data layers, where the cleaned and transformed weather data is stored for machine learning training.

4.4.3 Final solution

Parquet was selected as the final storage format for the weather analysis pipeline. Based on all reasons above and the fact that Parquet files are much easier to work with than ORC and Avro files.

4.4.4 Key takeaways

Write vs. Read: While Avro is better for fast writing (ingestion), Parquet is significantly superior for the heavy analytical reading required by weather forecasting.

Cost Efficiency: By using Parquet, we reduce cloud storage costs due to higher compression and lower compute costs because queries finish faster.

Ecosystem Compatibility: Parquet acts as a "universal language," allowing our weather data to be easily used by Spark, Pandas Python, and BI tools without complex conversion.

4.5 System Integration

4.5.1 Problem Description

Letting Kafka send data to Spark component. Kafka (on k8s) needs configuration to send to Spark. Or Spark to Elasticsearch.

4.5.2 Approaches tried

Dockerfile configuration: We use docker-compose.yml to configure the Kafka to enable connection between Kafka and Spark. Same for Spark to Elasticsearch.

Kubernetes .yaml file: We config in .yaml files. Basically, it is like docker-compose.yml, but for k8s.

4.5.3 Final solution

Since deploying the system on k8s is a requirement, so we chose to use .yaml files to write all configurations details.

4.5.4 Key takeaways

Kubernetes-aware Integration: Using Kubernetes service discovery is essential for stable communication between Kafka and Spark in dynamic container environments.

Avoid Hard-coded Endpoints: Static IP-based configurations are unreliable in containerized systems and should be avoided.

4.6 Performance Optimization

4.6.1 Problem Description

Handling data coming from S3 for each ML inference.

4.6.2 Approaches tried

Caching data: Caching data inside a Spark DataFrame. We only require the previous 24-hour data to load at the start of the hour (For e.g. if the time now is 3:01, then for the next 58 minutes, no historical data will be loaded. The data will only be loaded after the time has reached 4:00)

4.6.3 Final solution

Caching data is the only solution. Therefore, it was chosen.

4.6.4 Key takeaways

It is better to load the data periodically, than to load it after every single minute.

4.7 Monitoring and Debugging

4.7.1 Problem Description

Return logs for monitoring.

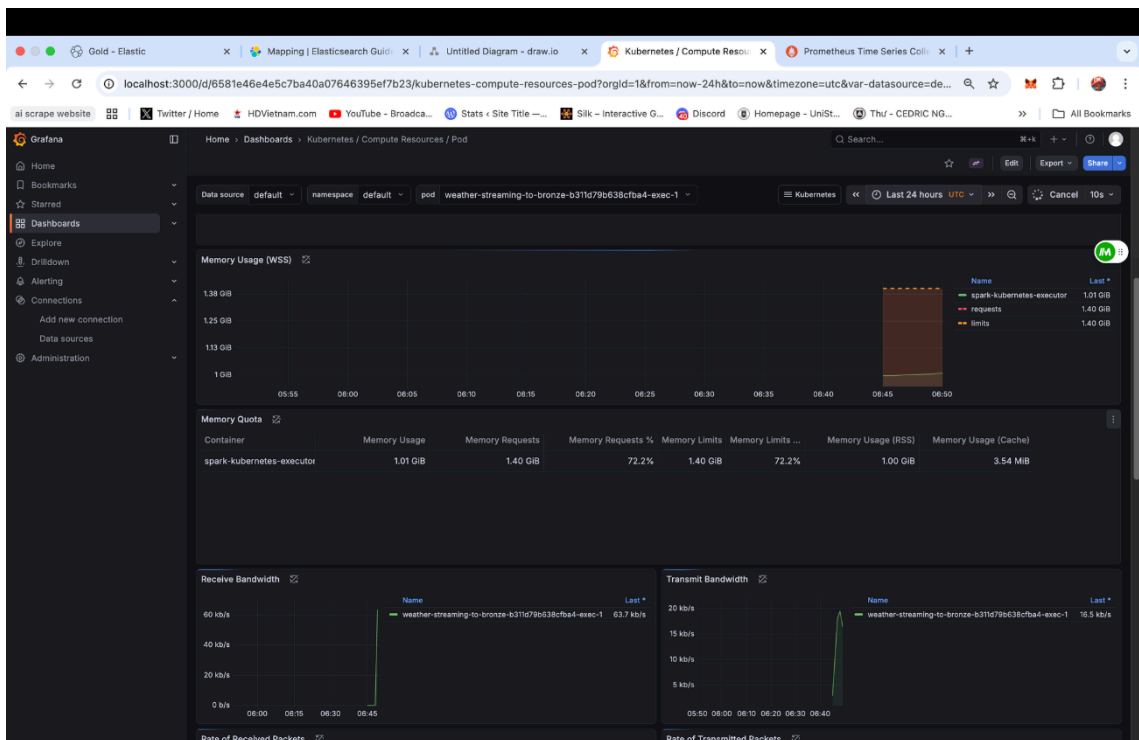
4.7.2 Approaches tried

Using pre-built command in k8s: (kubectl describe node data-platform-control-plane | egrep -A3) to show all logs.

Using Grafana: Implement Grafana to monitor the system

4.7.3 Final solution

We use k8s command because Grafana is too heavy for our system. Below is an image we try to run Grafana.



4.7.4 Key takeaways

Kubernetes-native Tools Are Effective: Built-in Kubernetes commands provide sufficient visibility for debugging and monitoring in small to medium-scale deployments.

Resource Monitoring Supports Debugging: Monitoring CPU and memory usage helps identify performance bottlenecks and resource misconfigurations.

4.8 Scaling

4.8.1 Problem Description

Increase number of Spark executors.

4.8.2 Approaches tried

To improve parallel data processing, the number of Spark executors was increased from a single executor to two executors. This horizontal scaling approach allows Spark to process multiple partitions of the incoming data stream concurrently, reducing processing delays and improving overall throughput.

4.8.3 Final solution

The final system configuration uses two Spark executors for the streaming application. This setup provides sufficient parallelism for the current workload while maintaining predictable resource consumption on the Kubernetes cluster.

4.8.4 Key takeaways

Incremental Scaling: Gradually increasing executors and testing performance helps identify the optimal configuration for the workload.

4.9 Data Quality & Testing

4.9.1 Problem description

Before pushing code changes to GitHub, it is necessary to test all system components locally to ensure correctness, stability, and integration compatibility. The system consists of multiple components, including data ingestion, processing, and storage services. Without proper local testing, errors may propagate into the shared repository, leading to broken pipelines and increased debugging effort during deployment.

4.9.2 Approaches tried

Individual components were tested independently in a local development environment. Unit testing was applied to verify core logic, while manual testing was used to validate data flow between components. Docker containers were used to simulate the production environment, allowing services such as Kafka and Spark to be tested locally before integration.

4.9.3 Final solution

The final approach combines local component testing with container-based integration testing. Each component is first tested independently to verify its functionality.

4.9.4 Key takeaways

Early Testing Reduces Errors: Testing components locally helps detect issues early in the development cycle.

Component Isolation Is Effective: Verifying each component separately simplifies debugging and improves reliability.

4.10 Fault Tolerance

4.10.1 Problem Description

Handling faults (for e.g. lose connections, Kafka broker(s) does not work, ...)

4.10.2 Approaches tried

Kill Spark application(s) when completed and generate new application via CronJob.

4.10.3 Final solution

Kill Spark application(s) when completed and generate old application based on the killed Spark application(s) via CronJob.

4.10.4 Key takeaways

Automated Recovery Is Essential: Using CronJobs to restart Spark applications enables hands-off recovery from common failures.